

Correction du Devoir de Compilation

Questions de Cours (6 points)

1. Qu'est-ce qu'un compilateur ?

Un compilateur est un programme informatique qui transforme du code source écrit dans un langage de programmation (langage haut niveau) en un code cible (généralement en langage machine ou code intermédiaire). Ce processus permet à un ordinateur d'exécuter le programme.

2. Définir les trois premières phases de la compilation.

Les trois premières phases de la compilation sont :

- **Analyse lexicale** : Découpe le code source en tokens (unités lexicales) comme les mots-clés, identificateurs, opérateurs, etc.
- **Analyse syntaxique** : Vérifie que la séquence de tokens respecte la grammaire du langage et construit un arbre syntaxique.
- **Analyse sémantique** : Vérifie la cohérence sémantique du programme (types, déclarations, etc.) et enrichit l'arbre syntaxique avec des informations supplémentaires.

3. Citer trois différences entre un compilateur et un interpréteur.

- Un **compilateur** traduit tout le programme avant son exécution, tandis qu'un **interpréteur** exécute le programme ligne par ligne.
- Un compilateur génère un fichier exécutable, alors qu'un interpréteur ne produit pas de fichier exécutable.
- Un compilateur est généralement plus rapide à l'exécution, mais plus lent à la compilation, tandis qu'un interpréteur est plus lent à l'exécution mais ne nécessite pas de phase de compilation séparée.

4. Citer et définir trois qualités d'un compilateur.

- **Correction** : Le compilateur doit produire un code cible qui sémantiquement équivalent au code source.
- **Efficacité** : Le code généré doit être optimisé pour une exécution rapide et une utilisation minimale des ressources.
- **Portabilité** : Le compilateur doit pouvoir être utilisé sur différentes plateformes et architectures.

5. Qu'est-ce qu'une grammaire ambiguë ?

Une grammaire est dite ambiguë si elle permet de générer une même chaîne de caractères (ou un même programme) avec plusieurs arbres syntaxiques différents. Cela pose problème car l'interprétation du programme peut alors être multiple.

Exercices : Automates et Expressions Régulières (7 points)

1. Répondre par Vrai ou Faux :

Un automate fini déterministe :

- a. **Vrai** : Par définition, un AFD ne peut pas être non déterministe.
- b. **Vrai** : Un AFD a exactement un seul état initial.
- c. **Faux** : Un AFD peut avoir plusieurs états finaux.

2. Automates pour $\Sigma = \{a, b\}$:

- a. **Mots avec un nombre de a multiple de 3 :**

L'automate a 3 états q_0, q_1, q_2 représentant respectivement 0, 1 et 2 a modulo 3. Les transitions sont :

- $q_0 \xrightarrow{a} q_1, q_1 \xrightarrow{a} q_2, q_2 \xrightarrow{a} q_0$
- Toutes les transitions b restent dans le même état.

L'état final est q_0 .

- b. **Mots avec un nombre de a multiple de 2 mais pas de 3 :**

On combine deux automates :

- Un automate pour "multiple de 2" (2 états).

- Un automate pour "non multiple de 3" (états q_0, q_1, q_2 comme ci-dessus, état final q_1, q_2).

Le produit de ces automates donne 6 états. Les états finaux sont ceux où le premier composant est q_0 (multiple de 2) et le second est q_1 ou q_2 (non multiple de 3).

3. Expression régulière pour l'automate donné :

L'automate fourni (non dessiné ici) semble avoir deux états avec des transitions :

- État 1 $\xrightarrow{1}$ État 1
- État 1 $\xrightarrow{0}$ État 2
- État 2 $\xrightarrow{1}$ État 2

L'expression régulière correspondante est : $1^*0(1)^*$.

4. Table de transition :

Pour l'automate précédent :

État	0	1
1	2	1
2	-	2

(Note : "-" indique une transition absente)

Exercice 2 : Grammaire du Langage Simple (7 points)

Voici la grammaire pour le langage décrit :

```

Programme → 'prog' Identificateur [Auteur] [Date] Corps 'fin'
Auteur → 'auteur' ':' Identificateur
Date → 'créé' 'le' ':' DateLiteral
Corps → 'DebutCorps' Declarations Instructions 'FinCorps'
Declarations → Declarations Declaration |
Declaration → 'variable' Identificateur ':' Type ';'
Type → 'entier'
Instructions → Instructions Instruction |
Instruction → Identificateur '=' Expression ';'
Expression → Expression Op Expression | Identificateur | Nombre
Op → '+' | '-' | '*' | '/'

```

Explications :

- Les éléments entre [] sont optionnels.
- Identificateur et Nombre sont des terminaux non définis ici (supposés conformes à Java).
- DateLiteral représente un format de date non spécifié.
- représente la production vide.
- La grammaire est récursive pour les déclarations et instructions pour permettre des listes.